3.2 `if` Statements

[This section corresponds to K&R Sec. 3.2]

The simplest way to modify the control flow of a program is with an `if` statement, which in its simplest form looks like this:

```
if(x > max)
        max = x;
```

Even if you didn't know any C, it would probably be pretty obvious that what happens here is that if `x` is greater than `max`, `x` gets assigned to `max`. (We'd use code like this to keep track of the maximum value of `x` we'd seen--for each new `x`, we'd compare it to the old maximum value `max`, and if the new value was greater, we'd update `max`.)

More generally, we can say that the syntax of an `if` statement is:

```
if( expression )
        statement
```

where *expression* is any expression and *statement* is any statement.

What if you have a series of statements, all of which should be executed together or not at all depending on whether some condition is true? The answer is that you enclose them in braces:

```
if( expression )
        {
        statement<sub>1</sub>
        statement<sub>2</sub>
        statement<sub>3</sub>
        }
```

As a general rule, anywhere the syntax of C calls for a statement, you may write a series of statements enclosed by braces. (You do not need to, and should not, put a semicolon after the closing brace, because the series of statements enclosed by braces is not itself a simple expression statement.)

An `if` statement may also optionally contain a second statement, the ``else` clause," which is to be executed if the condition is not met. Here is an example:

```
if(n > 0)
        average = sum / n;
else    {
        printf("can't compute average\n");
        average = 0;
        }
```

The first statement or block of statements is executed if the condition *is* true, and the second statement or block of statements (following the keyword `else`) is executed if the condition is *not* true. In this example, we can compute a meaningful average only if `n` is greater than 0; otherwise, we print a message saying that we cannot compute the average. The general syntax of an `if` statement is therefore

```
if( expression )
        statement<sub>1</sub>
else
```

```
        statement<sub>2</sub>
```

(where both *statement*`<sub>`*1*`</sub>` and *statement*`<sub>`*2*`</sub>` may be lists of statements enclosed in braces).

It's also possible to nest one `if` statement inside another. (For that matter, it's in general possible to nest any kind of statement or control flow construct within another.) For example, here is a little piece of code which decides roughly which quadrant of the compass you're walking into, based on an `x` value which is positive if you're walking east, and a `y` value which is positive if you're walking north:

```
if(x > 0)
        {
        if(y > 0)
                printf("Northeast.\n");
        else    printf("Southeast.\n");
        }
else    {
        if(y > 0)
                printf("Northwest.\n");
        else    printf("Southwest.\n");
        }
```

When you have one `if` statement (or loop) nested inside another, it's a very good idea to use explicit braces `{}`, as shown, to make it clear (both to you and to the compiler) how they're nested and which `else` goes with which `if`. It's also a good idea to indent the various levels, also as shown, to make the code more readable to humans. Why do both? You use indentation to make the code visually more readable to yourself and other humans, but the compiler doesn't pay attention to the indentation (since all whitespace is essentially equivalent and is essentially ignored). Therefore, you also have to make sure that the punctuation is right.

Here is an example of another common arrangement of `if` and `else`. Suppose we have a variable `grade` containing a student's numeric grade, and we want to print out the corresponding letter grade. Here is code that would do the job:

```
if(grade >= 90)
        printf("A");
else if(grade >= 80)
        printf("B");
else if(grade >= 70)
        printf("C");
else if(grade >= 60)
        printf("D");
else    printf("F");
```

What happens here is that exactly one of the five `printf` calls is executed, depending on which of the conditions is true. Each condition is tested in turn, and if one is true, the corresponding statement is executed, and the rest are skipped. If none of the conditions is true, we fall through to the last one, printing ``F''.

In the cascaded `if`/`else`/`if`/`else`/... chain, each `else` clause is another `if` statement. This may be more obvious at first if we reformat the example, including every set of braces and indenting each `if` statement relative to the previous one:

```
if(grade >= 90)
        {
        printf("A");
```

```
        }
else    {
        if(grade >= 80)
                {
                printf("B");
                }
        else    {
                if(grade >= 70)
                        {
                        printf("C");
                        }
                else    {
                        if(grade >= 60)
                                {
                                printf("D");
                                }
                        else    {
                                printf("F");
                                }
                        }
                }
        }
```

By examining the code this way, it should be obvious that exactly one of the `printf` calls is executed, and that whenever one of the conditions is found true, the remaining conditions do not need to be checked and none of the later statements within the chain will be executed. But once you've convinced yourself of this and learned to recognize the idiom, it's generally preferable to arrange the statements as in the first example, without trying to indent each successive `if` statement one tabstop further out. (Obviously, you'd run into the right margin very quickly if the chain had just a few more cases!)

Read sequentially: prev next up top